Animals TCG – Projet Java

Jeu de cartes à collectionner avec l'API iNaturalist



Adrien (GitHub BanADrien) et **Léo** (GitHub leochris8)

SDV – Cours de Java – Octobre 2026

Dépôt : **https://github.com/BanADrien/Animals-TCG**

Sommaire

1. Présentation du projet
2. L'API utilisée : iNaturalist
3. Les endpoints de notre application
4. Notre valeur ajoutée
5. Architecture et base de données
6. Les collections Java (List, Set, Map)
7. Les tests
8. Docker et lancement du projet
9. Organisation du travail en binôme
10. Conclusion

1. Présentation du projet

Animals TCG est un jeu de cartes à collectionner (TCG = *Trading Card Game*) sur le thème des animaux. L'idée vient des cartes Pokémon : au lieu de créatures inventées, chaque carte représente un **vrai animal**, avec sa vraie photo, son nom français, son nom scientifique et sa classe (oiseau, insecte, mammifère…). Toutes ces informations viennent de l'API publique d'**iNaturalist**, un site où des naturalistes du monde entier partagent leurs observations.

Le principe du jeu

- Le joueur **crée un compte** (pseudo + mot de passe) et reçoit **100 pièces**.
- Il dépense **50 pièces** pour **ouvrir un booster** : 5 cartes tirées au hasard dans l'API.
- Chaque carte a une **rareté** (Commune, Peu commune, Rare, Légendaire) qui dépend du nombre de fois où l'animal a été observé dans la nature : plus il est observé, plus il est commun.
- Les cartes sont rangées dans sa **collection**, où il peut chercher, filtrer, mettre des cartes en favori et **revendre ses doublons** (10 pièces) pour racheter des boosters.
- Une page de **statistiques** résume sa collection (nombre d'espèces, répartition par rareté et par type, doublons).



Figure 1 – Page d'accueil du jeu

Technologies utilisées

Technologie	Utilisation dans le projet
Java 27	Langage du projet
Spring Boot 4.1.1	Serveur web, contrôleurs, injection des services
Spring Data JPA / Hibernate	Enregistrement des joueurs et des cartes, méthodes de recherche
Thymeleaf	Pages HTML générées côté serveur
MySQL 8.4 + Docker Compose	Base de données lancée dans un conteneur
RestClient + Jackson 3	Appel de l'API iNaturalist et lecture du JSON
BCrypt (spring-security-crypto)	Hachage des mots de passe
JUnit 6, MockMvc, H2	Tests unitaires et tests des pages
Maven (wrapper `mvnw`), Git, GitHub	Compilation, versions, travail à deux

2. L'API utilisée : iNaturalist

iNaturalist propose une API REST gratuite, **sans clé**. Nous utilisons l'endpoint qui liste les *taxons* (les espèces vivantes) :

```
GET https://api.inaturalist.org/v1/taxa
```

L'exemple fourni dans la consigne, `/v1/taxa?q=fox`, cherche les taxons dont le nom contient « fox ». Le résultat mélange des animaux, des plantes et des familles entières : pour un jeu de cartes, il fallait donc filtrer. Voici les paramètres que notre application envoie vraiment :

Paramètre	Rôle
`taxon_id=1`	Uniquement les descendants du règne **Animalia** (pas de plantes ni de champignons)
`rank=species`	Uniquement des espèces (pas de familles ni de genres)
`locale=fr`	Le nom courant est renvoyé en français (« Renard roux » au lieu de « Red Fox »)
`per_page=50`	50 animaux par page
`order_by=observations_count` `order=desc`	Tri du plus observé au moins observé (c'est la base de notre système de rareté)
`page=…`	Numéro de la page, choisi au hasard par notre tirage

Exemple d'appel fait par l'application :

```
https://api.inaturalist.org/v1/taxa?taxon_id=1&rank=species&locale=fr&per_page=50
    &order_by=observations_count&order=desc&page=37
```

Les champs du JSON que nous utilisons

La réponse contient beaucoup d'informations ; nous ne gardons que celles utiles à une carte (réponse simplifiée) :

```
{
  "total_results": 774544,
  "results": [
    {
      "id": 42069,                                  → identifiant de l'animal (inatId)
      "name": "Vulpes vulpes",                      → nom scientifique
      "preferred_common_name": "Renard roux",       → nom français
      "iconic_taxon_name": "Mammalia",              → classe, traduite en « Mammifère »
      "observations_count": 160750,                 → sert à la rareté
      "default_photo": {
        "medium_url": "https://static.inaturalist.org/photos/265916780/medium.jpg"
      }
    }
  ]
}
```

Limite de l'API : en parcourant les pages, l'API refuse d'aller au-delà de **10 000 résultats** (page × per_page ≤ 10 000), soit 200 pages de 50. Notre jeu tire donc ses cartes parmi les **10 000 espèces animales les plus observées**, ce qui est largement suffisant et garantit des animaux qui ont presque tous une photo.

3. Les endpoints de notre application

Notre application est un site web (Spring MVC + Thymeleaf) : chaque URL renvoie une page HTML ou, pour les formulaires (POST), fait une action puis redirige vers une page. Le joueur connecté est retenu dans la **session HTTP** (attribut `joueurId`) ; sans session, les pages protégées redirigent vers `/connexion`.

Méthode	URL	Rôle	Connexion
GET	`/`	Page d'accueil	Non
GET	`/inscription`	Formulaire d'inscription	Non
POST	`/inscription`	Crée le compte (mot de passe haché, 100 pièces) et connecte le joueur	Non
GET	`/connexion`	Formulaire de connexion	Non
POST	`/connexion`	Vérifie le pseudo et le mot de passe, ouvre la session	Non
GET	`/deconnexion`	Ferme la session	—
GET	`/profil`	Pseudo et pièces du joueur, formulaires de modification	Oui
POST	`/profil/pseudo`	Change le pseudo (refusé s'il est déjà pris)	Oui
POST	`/profil/mot-de-passe`	Change le mot de passe (l'ancien doit être correct)	Oui
POST	`/profil/supprimer`	Supprime le compte et ses cartes	Oui
GET	`/booster`	Page booster : pièces restantes, prix, cartes obtenues	Oui
POST	`/booster/ouvrir`	Tire 5 cartes, les enregistre, retire 50 pièces	Oui
GET	`/collection`	Collection du joueur, avec filtres `recherche`, `type`, `rarete`, `favoris`	Oui
GET	`/collection/{id}`	Détail d'une carte (seulement si elle appartient au joueur)	Oui
POST	`/collection/{id}/favori`	Ajoute ou retire la carte des favoris	Oui
POST	`/collection/{id}/vendre`	Vend un doublon pour 10 pièces (refusé si c'est le seul exemplaire)	Oui
GET	`/statistiques`	Statistiques de la collection	Oui

4. Notre valeur ajoutée

L'API iNaturalist est une base de données scientifique : elle donne des listes d'espèces, rien de plus. Notre travail a été de la **transformer en jeu**, avec des règles, une progression et une interface. Voici ce que nous avons ajouté par rapport à un simple affichage des résultats de l'API.

4.1 Une rareté basée sur la nature réelle

Nous n'avons pas inventé la rareté des cartes : elle vient du **nombre d'observations** réelles de l'animal. Comme l'API trie les animaux du plus observé au moins observé, les premières pages contiennent les animaux les plus communs (le Canard colvert, près d'un million d'observations) et les dernières les plus rares. Le tirage se fait en deux étapes (classe `TirageService`) :

1. on tire un nombre entre 0 et 99 pour choisir la **rareté** ;
2. on choisit une **page au hasard** dans la tranche de cette rareté, puis un animal au hasard dans la page.

Rareté	Chance	Pages de l'API	Observations (environ)	Symbole
Commune	60 %	1 à 10	plus de 50 000	●
Peu commune	25 %	11 à 50	15 000 à 50 000	◆
Rare	12 %	51 à 120	4 000 à 15 000	★
Légendaire	3 %	121 à 200	2 000 à 4 000	★★

```
// 60% Commune, 25% Peu commune, 12% Rare, 3% Legendaire
public String tirerRarete(int nombre) {
    if (nombre < 60) {
        return "Commune";
    } else if (nombre < 85) {
        return "Peu commune";
    } else if (nombre < 97) {
        return "Rare";
    } else {
        return "Légendaire";
    }
}
```

Le hasard (`random.nextInt(100)`) est séparé de la règle (`tirerRarete(int)`) : la règle peut donc être testée avec des valeurs précises.

4.2 L'ouverture de booster

- Un booster contient **5 animaux différents** : un `Set` des identifiants déjà tirés empêche les doublons dans un même booster, et le tirage s'arrête après 20 essais pour ne jamais boucler à l'infini.
- Les cartes sont tirées **avant** de retirer les pièces : si l'API ne répond pas, le joueur ne perd rien et voit le message « Impossible de contacter l'API ».
- Après l'ouverture, la page redirige (`POST` puis `GET`) : actualiser la page n'achète pas un deuxième booster.
- Les cartes apparaissent **une par une** avec une petite animation, comme quand on ouvre un vrai paquet.



Figure 2 – Un booster ouvert : 2 Communes (vert), 2 Peu communes (bleu) et 1 Légendaire (or)

4.3 Une petite économie de jeu

Le joueur commence avec 100 pièces, un booster coûte 50 pièces et un doublon se revend 10 pièces. On ne peut donc pas ouvrir des boosters à l'infini : il faut gérer ses pièces et revendre ses doublons. Ces valeurs ne sont pas écrites dans le code mais dans `application.properties` (`jeu.pieces.depart`, `jeu.booster.prix`, `jeu.vente.doublon`), ce qui permet de les changer sans recompiler.

4.4 Des cartes au style Pokémon rétro

Toute l'interface reprend les menus des jeux Pokémon sur Game Boy Advance : boîtes de dialogue au cadre épais, polices pixel, flèche ▶ au survol des menus, flèche ▼ qui clignote dans les messages. Chaque rareté a sa couleur (vert, bleu, violet, or) et son symbole, et les cartes **légendaires** ont un halo doré et un **reflet animé** comme une carte holographique.

4.5 Une collection utile, avec des statistiques

- Recherche par nom (français ou scientifique), filtres par type, par rareté et favoris, qui peuvent se combiner.
- Page de détail d'une carte, favoris, vente des doublons uniquement (on ne peut pas perdre son dernier exemplaire).
- Statistiques : nombre de cartes et d'espèces différentes, barres façon « barres de PV » par rareté et par type, liste des doublons.

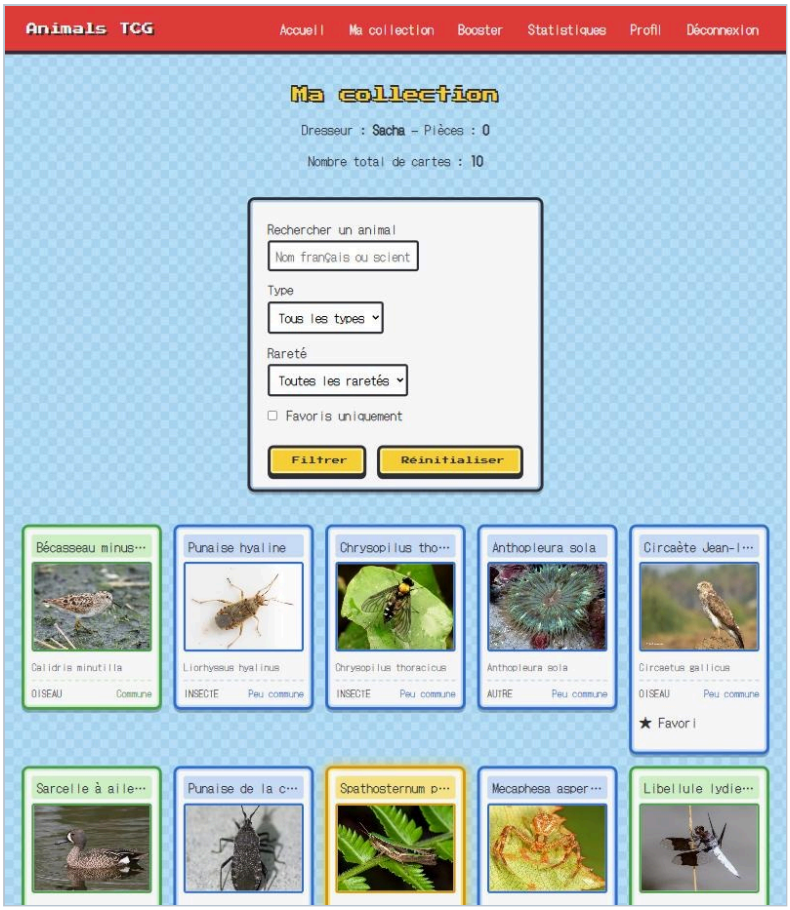


Figure 3 – La collection avec les filtres (recherche, type, rareté, favoris)



Figure 4 – Détail d'une carte mise en favori

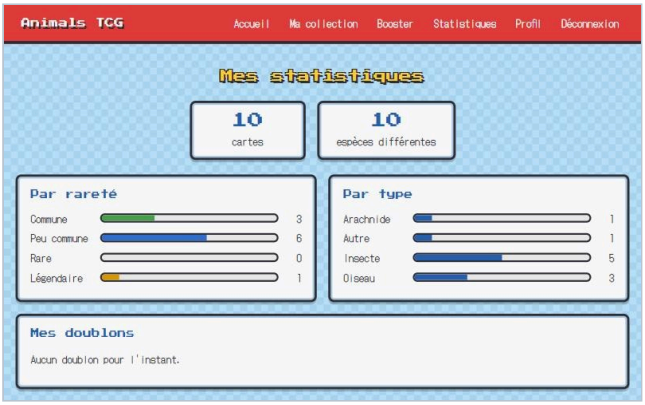


Figure 5 – Statistiques avec les barres façon PV

4.6 Des comptes sécurisés

- Les mots de passe sont **hachés avec BCrypt** : en base, on ne voit que `$2a$10$…`, jamais le mot de passe.
- Le pseudo est unique (contrainte en base + vérification dans le service).
- Le joueur connecté est gardé dans la session ; toutes les pages de jeu vérifient la session.
- Une carte ne peut être affichée, mise en favori ou vendue **que par son propriétaire** : nous avons vérifié qu'un autre joueur qui tape l'URL d'une carte est renvoyé vers sa propre collection.



Figure 6 – Inscription (`GET /inscription`)

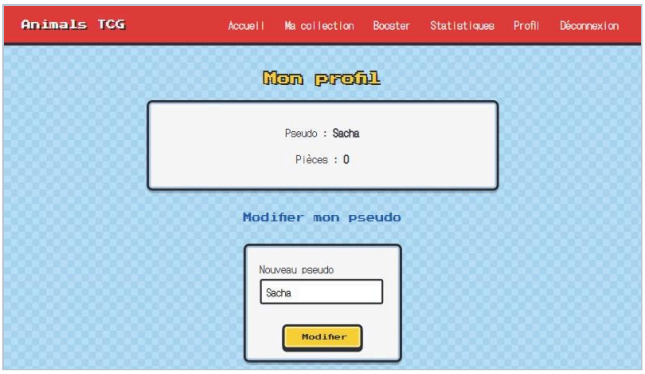


Figure 7 – Profil : pseudo, pièces et modification du compte

4.7 Des données propres et en français

- Les animaux **sans photo** sont écartés : une carte sans image n'aurait pas d'intérêt.
- Si l'API ne connaît pas de nom français, la carte affiche le **nom scientifique**.
- La classe latine est traduite : *Aves* → Oiseau, *Insecta* → Insecte, *Mammalia* → Mammifère…
- **Aucun animal n'est écrit en dur** dans le code : tout vient de l'API, puis de notre base MySQL.

5. Architecture et base de données

Organisation du code

```
src/main/java/fr/sdv/animalstcg/
├── models/          Joueur, Carte (entités JPA)
├── repositories/    JoueurRepository, CarteRepository (Spring Data JPA)
├── services/        InaturalistService (appel API), TirageService (hasard),
│                    BoosterService (ouverture), StatistiquesService, JoueurService (comptes)
└── controllers/     Accueil, Compte, Booster, Collection, Statistiques
src/main/resources/
├── templates/       pages Thymeleaf (accueil, inscription, booster, collection, carte...)
├── static/css/      style.css (style commun), booster.css, statistiques.css
└── application.properties
```

Les contrôleurs reçoivent les requêtes et préparent les pages, les services contiennent la logique (appel de l'API, tirage, calculs) et les repositories parlent à la base de données.

Les tables

Hibernate crée les tables tout seul à partir des entités (`spring.jpa.hibernate.ddl-auto=update`). Un joueur possède plusieurs cartes : relation `@ManyToOne` de `Carte` vers `Joueur`. Une ligne de la table `carte` correspond à **une carte obtenue** : tirer deux fois le même animal crée deux lignes, ce qui permet de gérer les doublons.

Table	Colonnes
joueur	id, pseudo (unique), mot_de_passe (haché), pieces
carte	id, inat_id, nom, nom_scientifique, type, rarete, photo_url, observations, favori, date_obtention, **joueur_id** (clé étrangère vers joueur)

Les recherches avec Spring Data JPA

Dans les repositories, nous n'écrivons que la signature des méthodes : Spring génère la requête SQL à partir du nom de la méthode. Quelques exemples du `CarteRepository` :

```
List<Carte> findByJoueurOrderByDateObtentionDesc(Joueur joueur);
List<Carte> findByJoueurAndType(Joueur joueur, String type);
List<Carte> findByJoueurAndRarete(Joueur joueur, String rarete);
List<Carte> findByJoueurAndFavoriTrue(Joueur joueur);
List<Carte> findByJoueurAndInatId(Joueur joueur, Long inatId);   // pour savoir si c'est un
doublon
long countByJoueur(Joueur joueur);

// requete JPQL ecrite a la main : cherche dans le nom francais ET le nom scientifique
@Query("SELECT c FROM Carte c WHERE c.joueur = :joueur AND (LOWER(c.nom) LIKE LOWER(CONCAT('%',
:texte, '%'))
        OR LOWER(c.nomScientifique) LIKE LOWER(CONCAT('%', :texte, '%')))")
List<Carte> chercherParNom(@Param("joueur") Joueur joueur, @Param("texte") String texte);
```

6. Les collections Java (List, Set, Map)

Collection	Où et pourquoi
List (`ArrayList`)	Partout où on manipule plusieurs cartes : les 50 animaux d'une page de l'API, les 5 cartes du booster, la collection du joueur, les résultats des filtres.
Set (`HashSet`)	Dans le booster, pour ne pas tirer deux fois le même animal (un Set refuse les doublons). Dans les statistiques, pour compter les **espèces différentes** : 3 Renards roux comptent pour 1 espèce.
Map (`HashMap` , `TreeMap` , `LinkedHashMap`)	Dans les statistiques : type → nombre de cartes (`TreeMap` , trié par ordre alphabétique), rareté → nombre de cartes (`LinkedHashMap` , qui garde l'ordre Commune → Légendaire), nom → nombre d'exemplaires pour les doublons.

Exemple : compter les cartes par type avec une `Map` (classe `StatistiquesService`) :

```
// type -> nombre de cartes (TreeMap pour avoir les types dans l'ordre alphabetique)
public Map<String, Integer> compterParType(List<Carte> cartes) {
    Map<String, Integer> resultat = new TreeMap<>();
    for (Carte carte : cartes) {
        String type = carte.getType();
        resultat.put(type, resultat.getOrDefault(type, 0) + 1);
    }
    return resultat;
}
```

Et le `Set` qui évite les doublons dans un booster (classe `BoosterService`) :

```
Set<Long> dejaTires = new HashSet<>();
...
if (dejaTires.contains(carte.getInatId())) {
    continue;   // animal deja dans ce booster, on retire
}
dejaTires.add(carte.getInatId());
```

7. Les tests

Le projet contient **31 tests**, qui passent tous avec `mvnw.cmd test`. Ils ne dépendent **ni d'Internet ni de Docker** :

- pour ne pas appeler la vraie API, nous utilisons un **faux service iNaturalist** (`FauxInaturalistService`), une sous-classe qui renvoie des animaux inventés ;
- quand un test a besoin d'une base de données, il utilise **H2**, une base en mémoire (`@DataJpaTest`) ;
- les pages sont testées avec **MockMvc** (`@WebMvcTest`) : Spring simule une requête HTTP, et nous remplissons la session à la main pour simuler un joueur connecté.

Classe de test	Tests	Ce qui est vérifié
InaturalistServiceTest	4	Lecture d'un JSON d'exemple : champs de la carte, nom scientifique si pas de nom français, animal sans photo ignoré, traduction des types
TirageServiceTest	4	Limites exactes des raretés (59 → Commune, 60 → Peu commune…), page toujours dans la bonne tranche, jamais au-delà de la page 200
TirageCartesTest	4	5 cartes différentes avec une rareté, pas de boucle infinie si l'API renvoie toujours le même animal
BoosterServiceTest	4	Avec H2 : 5 cartes enregistrées pour le joueur, 50 pièces retirées, booster refusé s'il n'y a pas assez de pièces
BoosterControllerTest	6	MockMvc : redirection sans connexion, pièces affichées, cartes transmises, messages d'erreur (pièces, API en panne), couleur selon la rareté
StatistiquesServiceTest	6	Espèces différentes (Set), comptes par type et par rareté (Map), doublons, collection vide, pourcentages des barres
StatistiquesControllerTest	3	MockMvc : redirection sans connexion, collection vide, vraies statistiques affichées

Exemple de test unitaire simple :

```
@Test
void rareteSelonLeNombreTire() {
    assertEquals("Commune", tirage.tirerRarete(59));
    assertEquals("Peu commune", tirage.tirerRarete(60));
    assertEquals("Rare", tirage.tirerRarete(96));
    assertEquals("Légendaire", tirage.tirerRarete(97));
}
```

8. Docker et lancement du projet

La base MySQL tourne dans un conteneur Docker, décrit dans `docker-compose.yml`. Personne n'a besoin d'installer MySQL sur son PC : une seule commande crée la base, l'utilisateur et le volume qui garde les données.

```
services:
  mysql:
    image: mysql:8.4
    container_name: animals-tcg-mysql
    environment:
      MYSQL_DATABASE: animals_tcg
      MYSQL_USER: tcg
      MYSQL_PASSWORD: tcg
    ports:
      - "3306:3306"
    volumes:
      - donnees-mysql:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-u", "tcg", "-ptcg"]
```

Lancer le projet

Prérequis : Java 27 et Docker Desktop.

```
git clone https://github.com/BanADrien/Animals-TCG.git
cd Animals-TCG
docker compose up -d          # demarre MySQL
mvnw.cmd spring-boot:run      # demarre l'application
# puis ouvrir http://localhost:8080

mvnw.cmd test                 # lance les 31 tests
```

La configuration (adresse de la base, URL de l'API, règles du jeu) est dans `src/main/resources/application.properties`.

9. Organisation du travail en binôme

Nous avons travaillé avec Git et GitHub en suivant une règle simple : **une branche par fonctionnalité**, puis une **Pull Request** vers `main`, vérifiée avant d'être fusionnée. Pour éviter les conflits, nous avons réparti le travail **par fichiers** et fixé dès le début ce qui était partagé : le nom de l'attribut de session (`joueurId`), les URL de chaque page et les règles du jeu.

Branche	Auteur	Contenu
`main` (début)	Adrien	Projet Spring Boot, docker-compose, entités `Joueur` et `Carte`, repositories, règles du jeu
`feature/gabarit`	Léo	Page d'accueil, menu commun, style Pokémon
`feature/booster`	Adrien	Service iNaturalist, tirage de la rareté, service et page booster
`feature/compte`	Léo	Inscription, connexion, BCrypt, profil
`feature/statistiques`	Adrien	Service et page de statistiques
`feature/collection`	Léo	Collection, filtres, détail d'une carte, favoris, vente des doublons

Avant chaque fusion, nous récupérions `main` pour vérifier que tout compilait et que les tests passaient encore avec le travail de l'autre.

10. Conclusion

Ce projet nous a permis de relier toutes les notions vues en cours dans une vraie application : appeler une API et lire son JSON, enregistrer des données avec JPA et faire des recherches avec Spring Data, utiliser les bonnes collections (List, Set, Map) selon le besoin, écrire des tests unitaires, lancer une base avec Docker Compose et travailler à deux avec Git (branches et Pull Requests).

Le plus intéressant a été de transformer des données scientifiques en règles de jeu, par exemple la rareté calculée à partir du nombre d'observations réelles.

Améliorations possibles :

- échanger des cartes entre joueurs ;
- des boosters par type (que des oiseaux, que des insectes…) ;
- un classement des joueurs selon le nombre d'espèces collectionnées ;
- une pièce gagnée chaque jour pour revenir jouer.